

Project topic 1: WhatsApp Group Wrapped (Python + Web)

WhatsApp Group Wrapped

Author - [Ansh Agrawal](#)

Project Specification

Python · JavaScript · HTML / CSS

1. Overview

This project involves building a browser-based, Spotify Wrapped-style slideshow that visualises social dynamics found in a WhatsApp group chat. The system has three parts: a Python chat generator, a Python statistics parser, and a JavaScript front-end. Python and JavaScript communicate solely through a JSON file that you design.

2. System Architecture

The pipeline flows as follows:

vocabulary.txt → chat.py → chat.txt → analyze.py → data.json → app.js → Browser

Suggested directory structure:

whatsapp-wrapped/

├── generator/chat.py

├── parser/analyze.py

└── web/ (index.html, style.css, app.js, *.html, *.css, *.js)

└── chat.txt

└── data.json

└── vocabulary.txt

3. Chat Generation (chat.py)

Write a Python script that generates a group chat from a provided vocabulary.txt file (comma-separated words), following this exact format:

DD/MM/YY, HH:MM - Sender_Name: Message text

The chat must contain clear, distinct personalities. Simple random word selection is insufficient. You must implement social behaviour patterns such as:

- Frequency weights: some users message far more often than others (e.g. a "Chatterbox")
- Time constraints: certain users are only active during specific hours (e.g. a "Night Owl" active 12am–4am)
- Message length: some users send one-liners while others write long paragraphs
- Reply streaks: one user sends several consecutive messages without any response (a "Ghoster" effect)

Use numpy functions where helpful. Each personality pattern must be clearly implemented and commented in the code. There should be at least 7 group members in the WhatsApp chat (not in your project team!). All of the (simulated) WhatsApp group members must have taken CS-108 this semester :-)

Run command:

```
python chat.py <vocabulary.txt>
```

4. Statistics Parser (analyze.py)

Write a Python parser that reads chat.txt, computes statistics, and outputs data.json. You must implement at minimum the following 10 statistics:

#	Statistic	Description
1	total_messages	Message count per person
2	total_words	Word count per person
3	night_owl	Most active sender between 12am – 4am
4	ghost	Person whose messages receive the fewest direct replies
5	conversation_starter	Most frequent sender after a long silence
6	most_used_emoji	Top 3 emojis per person
7	busiest_day	Single day with the most messages
8	longest_silence	Longest gap with zero messages (in hours/days)
9	avg_response_time	Median minutes before someone replies to each person
10	hype_person	Person with the fastest average reply time

In addition, you must define and implement at least one custom statistic that is specific to your group chat - something that a generic tool would not compute. Explain your reasoning for this metric in the report and in inline comments.

Important: several terms are intentionally left undefined. You must decide and document your own definitions for:

- What counts as a "direct reply" to a message?
- What gap in time defines a "long silence" for the conversation_starter metric?
- What makes a day "busy" vs "chaotic"?

State each definition in your code (as a comment) and justify your choice in the report.

Run command:

```
python analyze.py <chat.txt>
```

Allowed Python modules: numpy, datetime, json, re, pathlib, sys, os, time.

If you want to use any other library, confirm from TAs. If that doesn't break the purpose of the assignment, it will get approved.

5. The Wrapped Experience (app.js / index.html)

Build a full-screen, click-to-advance browser slideshow. Data is read only from data.json. All visual design (colour scheme, typography, animations, transitions) is your responsibility.

5.1 Global Slides

Each of the required statistics must have its own slide with an appropriate visualisation (e.g. bar chart, timeline, emoji grid). At minimum, think carefully about which chart type best represents each statistic and why.

5.2 Per-Person Profile Slides

The slideshow must include a dedicated profile for each member of the group chat. Each profile slide should display that person's individual statistics in one place, including:

- Total messages and total words sent
- Their activity heatmap or time-of-day distribution (bar chart or similar)

- Their top 3 most-used emojis
- Their average response time
- Any other relevant per-person metrics your parser computes

The per-person slides should be visually consistent with the rest of the slideshow but clearly distinguishable as a "profile" view. Add a functionality that lets the user choose whose profile they want to view. The user may wish to view multiple profiles at once.

6. data.json Schema

You must design the JSON schema before writing either the parser or the front-end. The schema should cleanly separate:

- Group-level statistics (one value per stat, e.g. busiest_day, longest_silence)
- Per-person statistics (a dictionary keyed by name, containing all per-person metrics)

The schema is a shared contract between analyze.py and app.js. Both sides must agree on it before implementation begins. Document the schema in your README.

7. Git & Collaboration Requirements

One project member should create a private git repository and share with the other team member. You can use either gitlab.com or github.com. Do NOT use public repository. All team members must contribute meaningfully to the repository:

- At least one substantial commit per team member per week (we will check this subjectively during the demo)
- At least one merge commit must be present in the final submission
- Commit messages must be descriptive and reflect the actual change made

8. Submission Checklist (in final gitlab/github repo)

8.1 Code

- chat.py runs from the command line with a vocabulary file path
- analyze.py runs from the command line with a chat file path
- data.json is generated correctly and matches what app.js expects
- The browser experience works on any machine with no hardcoded absolute paths
- Per-person profile slides are implemented and navigable

8.2 Documentation

- README explaining how to run the project and describing the data.json schema
- Inline comments in analyze.py for every non-obvious decision or definition

8.3 Report (LaTeX)

The report must include:

1. A list of external libraries used and their purpose
2. A description of all features implemented
3. An explanation of your custom statistic and why you chose it
4. The definition choices made in the parser (e.g. what counts as a reply, what is a silence), with justification
5. Hurdles faced and how you overcame them
6. What you would do differently with 10 more hours.
7. Bibliography with references and links to external sources
8. Write a suitable Makefile to compile the latex report

9. Marks Distribution (Total=18)

Category	Marks
Proper Git commits	2
Functionality of code	5
Report + Makefile	2
Code quality (modularity, inline comments)	2
Viva	4
Ingenuity	3

Project topic 2: Mini Game Hub (bash + Python)

Mini Game Hub

Author - [Shanmathi Vasudevan](#)

Project Specification

Bash · Python · Pygame

1. Overview

Build a secure, multi-user game hub that integrates Bash scripting for authentication and Python (Pygame) for gameplay. Two authenticated players select a game from a menu, play via a graphical interface, and have their results recorded on a persistent leaderboard.

2. System Architecture

The hub must start from a single entry point:

bash main.sh

Suggested directory structure:

hub/

|—— main.sh

|—— game.py

|—— leaderboard.sh

|—— games/ (tictactoe.py, othello.py, connect4.py)

|—— users.tsv

|—— history.csv

3. Authentication (main.sh - Bash)

main.sh handles user authentication before launching the game engine. It must:

- Prompt for two distinct usernames and passwords
- Hash passwords with [SHA-256](#) before storing them in users.tsv - never store plaintext
- On login, hash the input and compare it against the stored hash. If the username exists but the password does not match, display a suitable message and prompt them to try again.
- If a username does not exist, ask whether the user wants to register and update users.tsv accordingly
- Once both players are authenticated, call game.py with both usernames as command-line arguments

python3 game.py <username1> <username2>

4. The Game Engine (game.py - Python / Pygame)

game.py receives the two authenticated usernames and manages the full Python-side flow: the game menu, gameplay, post-game recording, and analytics.

4.1 Class Structure (Required)

You must define a base class representing a generic 2-player, turn-based board game. It must include properties and methods common to all such games, for example:

- Player names and whose turn it is
- The game board (as a NumPy array)
- A method to switch turns
- An abstract or overridable method to check for a win condition

Each individual game must be implemented in its own class that inherits from this base class, in a separate file under games/.

4.2 Required Games

The menu must offer at least the following three games. All games must be rendered in a Pygame GUI window; text-only gameplay is not permitted.

Game 1 - Tic-Tac-Toe (Expanded)

- Played on a 10×10 board; players need 5 marks in a row to win
- The winning line can be horizontal, vertical, or diagonal
- Players alternate placing X or O in any empty cell

Game 2 - Othello (Reversi)

- Played on an 8×8 board, starting with two Black and two White discs in the centre
- A move is valid only if it traps one or more opponent discs in a straight line between the newly placed disc and an existing own disc
- All trapped discs are flipped to the current player's colour; multiple lines can be flipped in one move
- If a player has no valid moves, their turn is skipped
- The player with more discs of their colour when no valid moves remain wins

Game 3 - Connect Four

- Played on a vertical 7×7 grid; coins fall to the lowest empty row in a column
- Players take turns dropping one coin into any column
- A player wins by getting 4 of their coins in a row (horizontal, vertical, or diagonal)
- The game ends in a draw if the board fills with no winner

You may add more games beyond these three.

4.3 Technical Constraints

- NumPy: All game boards must be represented as NumPy arrays. Win conditions must be checked using array operations (e.g., slicing to check for 4-in-a-row), not manual loops.
- Pygame: Every game must have a fully functional GUI. No terminal-based gameplay.

5. Analytics & Leaderboard

5.1 Recording Results (Python)

After each game concludes, `game.py` must append a row to `history.csv` containing: Winner, Loser, Date, and Game name.

5.2 Leaderboard Display (`leaderboard.sh` - Bash)

`game.py` must call `leaderboard.sh` after each game. The script reads `history.csv` and prints a formatted table to the terminal showing, per game, per user: number of wins, number of losses, and Win/Loss ratio, sorted by one of these metrics. Add an option in the GUI to allow the user to choose which metric they want the leaderboard sorted by, and pass this to `leaderboard.sh`

5.3 Visualisation (Matplotlib)

After the leaderboard is shown, game.py must display some Matplotlib charts, for example:

- A bar chart of the Top 5 Players by total win count
- A pie chart of the Most Played Games by frequency in history.csv

5.4 Post-Game Loop

After displaying the statistics, prompt both players: play another game or quit. If they continue, return to the game menu. If they quit, the program must terminate cleanly.

6. Allowed Modules

pygame-ce, numpy, matplotlib, pathlib, sys, os, time

If you want to use any other library, confirm from TAs. If that doesn't break the purpose of the assignment, it will get approved.

7. Git & Collaboration Requirements

One project member should create a private git repository and share with the other team member. You can use either gitlab.com or github.com. Do NOT use public repository. All team members must contribute meaningfully to the repository:

- At least one substantial commit per team member per week (we will check this subjectively during the demo)
- At least one merge commit must be present in the final submission
- Commit messages must be descriptive and reflect the actual change made

8. Submission Checklist

8.1 Code

- `bash main.sh` starts the program with no additional arguments
- `main.sh` authenticates both players and passes usernames to `game.py`
- `game.py` runs the menu, games, result recording, leaderboard call, and visualisation
- `users.tsv` is generated correctly by `main.sh` (SHA-256 hashed passwords)
- `history.csv` is generated correctly by `game.py` after each game
- No hardcoded absolute paths; the project works on any machine
- Your code should be modular: Keep different functionalities in separate scripts for better readability. Use functions and classes wherever required.

8.2 Documentation

- README explaining how to run the project
- Inline comments in all code files explaining the logic of each function or non-obvious block

8.3 Report (LaTeX)

The report must include:

1. A list of external libraries used and their purpose
2. A description of all features implemented
3. An explanation of the code logic for the generic class, each game, the authentication system, and the leaderboard
4. Hurdles faced and how you overcame them
5. What you would do differently with 10 more hours
6. Bibliography with references and links to external sources
7. Write a suitable Makefile to compile the latex report

9. Marks Distribution (Total=18)

Category	Marks
Proper Git commits	2

Functionality of code	5
Report + Makefile	2
Code quality (modularity, inline comments)	2
Viva	4
Ingenuity	3

10. References

- SHA-256 hashing in Bash: use the sha256sum command
 - Python inheritance: docs.python.org/3/tutorial/classes.html
 - Pygame CE official documentation: [pyga.me](https://www.pyga.me)
 - Othello rules: worldothello.org
 - Connect Four rules: boardgamegeek.com/boardgame/2719/connect-four
 - Sample modular project: github.com/rennaMAhcuS/Maze
 - Rules for the games mentioned:
1. [Tic-tac-toe](#)
 2. [Othello \(reversi\)](#)
 3. [Connect4](#)

Project topic 3: A Snake Stack (bash + Web)

A Snake Stack 

Author - [Sushant Padha](#)

Project Specification

JavaScript · Python (Flask) · Bash

1. Overview

The aim of the project is to build a browser-based Snake game with a full three-layer architecture: a JavaScript frontend that the player interacts with, a Python (Flask) backend that receives and stores game results, and a Bash administration script for inspecting and managing stored data from the terminal.

Each layer has a single, well-defined responsibility and must not reach into the concerns of another layer. JavaScript does not know how scores are saved. The Bash script does not know how the game works. Flask sits in between and connects them.

 This project is intentionally scoped to use a flat text file for storage instead of a database. This is not a limitation - it is a constraint that forces you to use Bash tools like awk, sed, and sort to process data.

2. System Architecture

The three layers communicate as follows:

Browser (JS) → POST /save_score → Flask (Python) → history.txt ← Bash script

Suggested directory structure:

snake-system/

|—— app.py

|—— admin.sh

|—— history.txt

|—— static/

|—— index.html (optionally make .js and .css files separately)

To start the system, run:

```
python3 app.py
```

Then open a browser and navigate to <http://127.0.0.1:5000>. The Flask server must only run on localhost.

3. Layer 1 - The Frontend (JavaScript + HTML5 Canvas)

The frontend is a single HTML file served by Flask. It is responsible for the entire game experience and for reporting results when the game ends.

3.1 Layout

The webpage layout must implement the following:

- A modal to display rules and ask for username at the start.
- A modal to display game over, cause, score, timestamp and time survived.
- Centrally placed HTML5 Canvas element for the actual game window.

- The website must follow a professional, modular layout.

Students are advised to look into the Bootstrap 5 framework for help with the styling. Other frameworks are also allowed, but individual fine-tuning of CSS is expected to create the aesthetic.

You must implement a set of consistent UI design and aesthetic choices on the page.

3.2 The Game

Implement a standard grid-based Snake game on a Canvas element, including:

- Snake controlled by arrow keys or WASD.
- Food appears at random empty cells when the snake eats food.
- Ensure at least 3 food types are implemented in the game:
Carrot (health +1); Pumpkin Pie (health +3); Golden Apple (immunity for 10 seconds)
- If not immune, the snake can die by: hitting a wall, or colliding with its own body.
- Game Over modal should mention the current user session's highest score.

NOTE: Score = health = length of snake

3.3 Reporting the Score

When the game ends, the frontend must automatically send the final score to the Flask server using the fetch API. This must happen without refreshing or navigating away from the page. The request should be a POST to /save_score with a JSON body, for example:

```
{ "name": "Herobrine", "score": 42, "cause": "WALL", "duration": 85 }
```

i Note: the JavaScript should only know where to send the score (the API endpoint URL). It must not contain any logic for how the score is stored, formatted, or processed - that is entirely the server's responsibility.

4. Layer 2 - The Backend (Python / Flask)

The Flask server has two responsibilities: serving the game to the browser, and receiving and storing scores.

4.1 Serving the Game

Flask must serve index.html when the user visits the root URL (/) in their browser. No other web server should be needed.

4.2 The Score Endpoint

Create a POST endpoint at /save_score. When it receives a valid request, it must:

- Parse the JSON payload to extract the score
- Validate the fields in the JSON payload to prevent corruption of data
- Record the score fields and timestamp by appending a new line to history.txt
- Return a JSON response confirming the save was successful

Each line written to history.txt must follow a consistent format so that the Bash script can parse it reliably. For example:

```
[2025-06-12 14:35:02] Herobrine | 42 | WALL | 85
```

The format is flexible, but it must be documented in your README and used consistently.

5. Layer 3 - The Administration Utility (Bash)

Write a bash script admin.sh for inspecting and managing the history.txt file. It must not depend on Python or Flask being running; it reads the flat file directly.

When run, the script must present a menu-driven interface with the following options:

- Select user name to query about.
- View recent games' scores, clearly formatted, in a paginated view.
- View analytics (mean score, mean time survived, fraction of wall deaths etc.) with optional filtering up to some timestamp or across the file.
- Delete entries based on timestamp, or user name, or entries that do NOT match the format. Should ask for confirmation.
- Perform log rotation (back up the original file, and keep only last 10 entries from history)
- View the file sorted by user name, or time stamp (default), or score.
- Exit

The script must handle edge cases gracefully: if history.txt does not exist or is empty etc.

Optional: Make the Bash script utilize ANSI escape codes for colored terminal output to categorize data; for example, 'Wall' deaths in blue, and 'Self' deaths in red etc.

i You must use Bash text-processing tools where appropriate: awk or cut for parsing fields, sort for ordering scores, sed for deleting specific lines, and tail or head for retrieving recent entries. Don't read the file with Python or any other language inside the Bash script.

6. Criteria

Your submission will be assessed against the following functional requirements:

- When the snake dies in the browser, the score must appear in history.txt quickly
- The Bash script must work correctly whether history.txt is empty, has one entry, or has many entries
- The JavaScript layer must contain no file I/O or storage logic - only the fetch call
- The Flask server must run successfully on localhost (127.0.0.1)

7. Git & Collaboration Requirements

One project member should create a private git repository and share with the other team member. You can use either gitlab.com or github.com. Do NOT use public repository. All team members must contribute meaningfully to the repository:

- At least one substantial commit per team member per week (we will check this subjectively during the demo)
- At least one merge commit must be present in the final submission
- Commit messages must be descriptive and reflect the actual change made

8. Checklist

8.1 Code

- `python3 app.py` starts the Flask server with no additional arguments
- The game is accessible at `http://127.0.0.1:5000` once the server is running
- Scores are written to `history.txt` automatically on game over
- `bash admin.sh` launches the administration menu
- No hardcoded absolute paths; the project works on any machine

8.2 Documentation

- README explaining how to run both the server and the admin script
- README documenting the exact format used in `history.txt`
- Inline comments in all files explaining non-obvious logic

8.3 Report (LaTeX)

The report must include:

1. A list of external libraries used and their purpose
2. A description of all features implemented
3. An explanation of how the three layers communicate, with reference to your code
4. The format chosen for `history.txt` and why
5. Hurdles faced and how you overcame them
6. What you would do differently with 10 more hours
7. Bibliography with references and links to external sources

8. Write a suitable Makefile to compile the latex report

9. Marks Distribution

Category	Marks
Proper Git commits	2
Functionality of code	5
Report + Makefile	2
Code quality (modularity, inline comments)	2
Viva	4
Ingenuity	3

10. References

- <https://flask.palletsprojects.com/en/stable/> : A resource for getting started with flask
- Flask Tutorial (Real Python)(<https://realpython.com/tutorials/flask/>)
- Flask Mega-Tutorial by Miguel Grinberg (<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-i-hello-world>)
- Snake ([https://en.wikipedia.org/wiki/Snake_\(video_game_genre\)](https://en.wikipedia.org/wiki/Snake_(video_game_genre)))
- About [food types](#).